

TwinaForms Data Flow, Authentication, Encryption, and Usage

Date: 2026-06-05

Package version: TwinaForms managed package 0.10.0.5

Subscriber package version id: 04tgL000000GburQAC

Purpose

This document explains how TwinaForms moves information between Salesforce, AWS, and public published forms. It covers the main usage flow, information flow, authentication model, encryption in transit and at rest, and Salesforce data touchpoints.

Basic Usage Instructions

1. A Salesforce admin installs the TwinaForms managed package.
2. The admin opens the TwinaForms app and goes to the Connect page.
3. The admin connects the subscriber org to TwinaForms through Salesforce OAuth.
4. TwinaForms verifies secure package service access between Salesforce and AWS.
5. The admin grants TwinaForms user access to selected Salesforce users.
6. The admin designs a form in Salesforce using the TwinaForms Designer.
7. The admin configures allowed Salesforce objects, fields, prefill behavior, and submit behavior.
8. The admin publishes the form.
9. Salesforce registers the published form policy with AWS and uploads the generated public form artifact.
10. End users open the public form URL, optionally prefill data, and submit the form.
11. AWS validates the published form request and performs tenant-scoped Salesforce API operations based on the server-side policy.
12. The Salesforce admin can review form records, publication status, and submission logs in the TwinaForms Salesforce app.

High-Level Information Flow

1. Package Setup and Connection

The subscriber admin starts in Salesforce. The packaged Connect page sends setup context to AWS, including the Salesforce org id, admin email, company name, and login base URL. This initial AWS connect

endpoint is intentionally public because it only starts the OAuth setup flow and accepts non-secret setup context.

AWS does not treat this pre-OAuth context as authentication, and it does not grant access to Salesforce data, AWS data, tenant secrets, or protected package service APIs. The admin then authorizes TwinaForms using Salesforce OAuth.

AWS receives the OAuth callback, exchanges the authorization code server-to-server, validates that the returned Salesforce org id matches the org that initiated setup, and stores the org-specific Salesforce connection in AWS Secrets Manager. If the org ids do not match, AWS rejects the connection.

After OAuth connection, AWS uses the authorized Salesforce session to retrieve or create a private per-org package service secret from the installed package. This lets Salesforce prove to AWS that later package service calls, such as connection verification and form publishing, came from the installed TwinaForms package in that subscriber org. The secret itself is protected trust material; each later signed request is short-lived and includes a timestamp, nonce, body hash, org id, and HMAC signature.

2. Form Design and Publish

The Salesforce package stores form design records in Salesforce custom objects. The admin configures form elements, prefill actions, submit actions, and allowed Salesforce object/field mappings.

When the admin publishes a form:

- Salesforce generates or uses the form id.
- Salesforce generates a per-form publish token.
- Salesforce sends AWS the server-side form policy and runtime definitions.
- AWS stores a hashed copy of the publish token with the server-side form policy in DynamoDB.
- Salesforce uploads or publishes the generated public form artifact to AWS-hosted form storage.

The published public form contains the form id and publish token needed to call the public prefill and submit endpoints. It does not contain Salesforce OAuth tokens, AWS secrets, tenant service secrets, or executable Salesforce command definitions.

3. Public Prefill

When a public user opens a form and a prefill request is triggered, the browser sends only the form id, publish token, and allowed lookup parameters to the AWS prefill endpoint.

AWS validates:

- the form id exists
- the incoming publish token matches the stored token hash
- the form is published
- the owning tenant is active
- the requested parameters are allowed by the server-side prefill policy
- the stored prefill commands and Salesforce objects are allowed

Only after those checks does AWS use the org-specific Salesforce connection to query the subscriber org. The browser cannot choose arbitrary Salesforce objects, fields, or SOQL.

4. Public Submit

When a public user submits a form, the browser sends the form id, publish token, and user-entered form values to the AWS submit endpoint.

AWS validates:

- the form id exists
- the incoming publish token matches the stored token hash
- the form is published
- the owning tenant is active
- the server-side submit policy allows the configured command types
- the configured Salesforce objects are allowed
- writable fields are allowed by policy
- higher-risk operations are allowed only by the configured form security mode

Only after those checks does AWS use the org-specific Salesforce connection to create, update, or otherwise interact with Salesforce according to the stored server-side submit definition.

5. Submission Logs

TwinaForms writes tenant-scoped submission log metadata to AWS for auditability and troubleshooting inside Salesforce. Basic metadata includes submission reference, form id, submitted time, outcome, failure category, and retention fields.

When detailed logs are enabled by plan and key configuration, submitted values and technical details are encrypted before storage. Plain log metadata does not include submitted values or raw Salesforce/API failure text.

Authentication Model

Salesforce OAuth

TwinaForms uses Salesforce OAuth authorization code flow for subscriber org connection. The subscriber admin authorizes TwinaForms from the packaged Connect page. AWS exchanges the authorization code server-to-server and validates the returned Salesforce org id.

AWS stores the resulting org-specific refresh token and instance URL in AWS Secrets Manager. The refresh token is used only server-side by AWS Lambda when TwinaForms needs to perform tenant-scoped Salesforce API operations.

Package-to-AWS Service Authentication

During OAuth setup, AWS uses the freshly authorized Salesforce OAuth access token to call a packaged Apex endpoint in the same subscriber org. That Apex endpoint is not a public AWS endpoint. Salesforce requires the OAuth access token, and Apex verifies that the requested orgId matches `UserInfo.getOrganizationId()` before returning or creating the private per-org signing secret.

The private per-org signing secret is protected trust material. It is stored in protected package-managed Salesforce storage and in the org-specific AWS connection secret. It is not exposed to the browser, public form HTML, normal Salesforce records, or subscriber users.

Later Salesforce package service calls to AWS are signed by Apex using HMAC. Each signed request includes the org id, timestamp, nonce, body hash, and signature. AWS verifies the signature, timestamp, body hash, and org context before accepting protected package service requests.

The 5-minute validation window applies to each later signed package-to-AWS request, not to the initial OAuth-time secret retrieval. AWS accepts an individual signed request only within that timestamp window and records each nonce so the same signed request cannot be replayed successfully after it has already been used.

This protects package service calls without exposing the signing secret to the browser or asking subscriber admins to create Named Credentials or External Credentials.

Public Form Authentication

Public forms do not use the tenant service secret or Salesforce OAuth token. Each published form uses a per-form publish token generated by the Salesforce package during publish.

AWS stores only a hashed copy of the publish token with the server-side form policy. On prefill and submit requests, AWS hashes the incoming token and compares it to the stored hash before proceeding.

The publish token is not sufficient by itself. AWS also validates form id, form status, tenant status, command allowlists, object allowlists, field allowlists, and form security mode.

Encryption and Data Protection

Encryption in Transit

TwinaForms uses HTTPS/TLS for network communication between:

- Salesforce packaged Apex and AWS Lambda endpoints
- AWS OAuth callback and Salesforce OAuth/token endpoints
- public browser forms and AWS prefill/submit endpoints
- browser users and published form hosting
- Salesforce users and Salesforce Lightning pages

Package-to-AWS service calls also include HMAC request signing in addition to HTTPS transport protection.

Token and Secret Storage

Salesforce OAuth refresh tokens are stored in AWS Secrets Manager in org-specific secrets. The central TwinaForms OAuth client configuration is also stored in AWS Secrets Manager.

The private per-org package service signing secret is stored in protected package-managed Salesforce storage and in the org-specific AWS connection secret. It is not exposed to the browser, public form HTML, normal Salesforce records, or subscriber users.

Data at Rest

AWS stores tenant records, form security policies, and submission log metadata in DynamoDB. DynamoDB provides encryption at rest through AWS-managed encryption. AWS Secrets Manager stores OAuth and connection secrets using AWS-managed secret storage controls.

Published public form artifacts are stored in S3 and served through CloudFront. These artifacts contain generated form UI and the per-form publish token, but they do not contain Salesforce OAuth tokens, tenant service secrets, AWS secrets, or backend executable command definitions.

Submission Log Detail Encryption

When detailed submission logs are enabled, sensitive submission details are encrypted before being stored in DynamoDB. The design uses a hybrid encryption model:

- Salesforce stores hidden org-level cryptographic material.
- AWS stores only the synced public key and key version.
- AWS encrypts detailed payloads with a one-time AES key.
- AWS encrypts the AES key with the org public key.
- The Salesforce Submission Logs UI decrypts detail using org-side private key material.

Plain metadata stores only operational fields such as outcome and failure category. Submitted values and raw Salesforce/API failure details are stored only in encrypted detail payloads when enabled.

Salesforce Data Touchpoints

Data Stored in Salesforce

The managed package stores form configuration in Salesforce custom objects, including:

- projects
- forms
- form versions
- form elements
- form actions
- publication records
- package configuration
- submission-log key metadata

Salesforce remains the design and administration surface for the solution.

Data Stored in AWS

TwinaForms stores limited Salesforce-related data in AWS:

- tenant metadata in DynamoDB
- org-specific Salesforce OAuth connection records in AWS Secrets Manager
- server-side form security policies and runtime definitions in DynamoDB
- generated public form artifacts in S3/CloudFront
- submission log metadata in DynamoDB
- encrypted submission log detail in DynamoDB when enabled

Data Used at Runtime but Not Stored in Public Artifacts

The public browser does not receive:

- Salesforce OAuth refresh tokens
- Salesforce access tokens
- AWS secrets
- tenant service secrets
- private per-org signing secrets
- executable Salesforce command definitions
- Salesforce object/field allowlists used for server enforcement

Those controls remain server-side in Salesforce package logic, AWS Secrets Manager, or DynamoDB policy records.

Data Flow Summary

1. Salesforce admin configures and publishes a form from the managed package.
2. Salesforce registers the form policy with AWS using signed package service access.
3. AWS stores the policy and hashed publish token.
4. AWS hosts the public form artifact.
5. A public user opens the form and submits data through HTTPS.
6. AWS validates form id, publish token, tenant status, and server-side policy.
7. AWS uses the org-specific Salesforce connection to perform allowed Salesforce API operations.
8. AWS writes tenant-scoped submission log metadata and encrypted detail when enabled.
9. Salesforce admins review records and logs inside the TwinaForms Salesforce app.